

What (cad) and (cad *) look like in the prover

Proof traces of verified cylindrical algebraic decision in PVS: the sequent as PVS prints it, the command, the strategy's messages, and the result, for twenty-one examples, plus the full step-by-step view of three proofs.

Reading the traces

Each example is a real PVS session on the `pvs_cad` library, fed from a script to `pvs -raw` and captured as printed. The typed commands are put back after each `Rule?` prompt (a scripted session does not echo them), and library-loading messages are cut; the other cuts and one change of indentation are listed in “About this document” at the end. Nothing else is changed.

Note (1 October 2026). These transcripts were recorded on 28 September 2026 in theories that import `cad_decide5`, so `(cad)` ran `decide5` and instantiated `decide5_correct`, and the times are those of that run. In a theory that imports `pvs_cad`, `(cad)` runs `decide8`, the Collins run, on prenex formulas over `real` and `decide_g` on formulas of any other shape, and instantiates `decide8_correct` or `decide_g_ok`; `(cad *)` there also takes formulas that are not prenex. The messages and times in such a theory can differ from those below.

The sequent

Formulas above the turnstile `|-----` are hypotheses (antecedents), numbered $-1, -2, \dots$; formulas below it are goals (consequents), numbered $1, 2, \dots$. The sequent claims: if every hypothesis holds, at least one goal holds. Braces `{1}` mark a formula that is new or changed by the last step; brackets `[1]` mark one carried over. Names such as `!CDG_1` or `cadsf_465` after a number are labels the strategies attach, so that they never refer to a formula by position.

(cad)

decides one closed formula: a prefix of `FORALL/EXISTS` over real variables, then any Boolean combination of polynomial comparisons. A true goal is proved; a false hypothesis closes the goal; otherwise the formula stays, labelled `cad`, with a message saying `TRUE` or `FALSE`.

(cad *)

works on the whole sequent. It keeps every formula it can use, turns other real terms (`sqrt(x)`, `f(x)`, `length(l)`) into unknown reals, forms $\forall \text{unknowns} : (\bigwedge \text{hypotheses}) \Rightarrow (\bigvee \text{goals})$, and decides that with `(cad)`.

What is trusted

Every step is checked by PVS. What remains to trust is the specification (that the semantics `fsem` and the theorem statements say what is intended), PVS itself (its kernel, and its ground evaluator running the executable decision procedure, in these transcripts `decide5`, whose soundness, `decide5_correct`, and completeness, `decide5_decides`, are proved in the library), and NASALib's saved proofs of the theorems the library imports, which were not replayed.

The examples

Times are PVS's own report (real time) for the whole proof, measured with `decide5` (theories importing `cad_decide5`) on 28 September 2026, on an M-series Mac, all in one session.

Lemma	What it says	Shape	Time	Result
(cad)				
<code>d_meet</code>	A circle meets a parabola	$\exists\exists$, degree 2	3.8 s	proved
<code>b_motzkin</code>	The Motzkin polynomial is nonnegative	$\forall\forall$, degree 6	1.1 s	proved
<code>b_folium</code>	The folium of Descartes reaches past (1, 1)	$\exists\exists$, degree 3	0.09 s	proved
<code>b_lemniscate</code>	The lemniscate fits in the disc of radius $\sqrt{2}$	$\forall\forall$, degree 4	1.1 s	proved
<code>c_cubic_root</code>	Every depressed cubic has a real root	$\forall\forall\exists$, degree 3	0.76 s	proved
<code>d_quad_suf</code>	The quadratic formula, as an existence theorem	$\forall\forall\forall\exists$, degree 3	2.1 s	proved
<code>h_wilk20</code>	Wilkinson's polynomial of degree 20	$\forall$, degree 20	2.9 s	proved
<code>h_cheb10</code>	Chebyshev T_{10} stays in $[-1, 1]$	$\forall$, degree 10	2.0 s	proved
<code>g_mignotte1</code>	A root located to within 10^{-5}	$\exists$, degree 10	1.3 s	proved
<code>m_quad_signs</code>	The sign chart of $x^2 - 5x + 6$	$\forall$, degree 2	1.6 s	proved
<code>c_sphere_in</code>	$x + y + z$ reaches 1.7 on the unit sphere	$\exists\exists\exists$, degree 2	11.6 s	proved
<code>t_forall_exists_forall</code>	Three alternating quantifiers	$\forall\exists\forall$, degree 4	22.9 s	proved
<code>d_no_inverse</code>	Not every real has an inverse	$\forall\exists$ hypothesis	0.54 s	proved
<code>u_wilk8_val</code>	A false claim is reported, not proved	$\exists$, degree 8	0.23 s	FALSE
(cad *)				
<code>s_disc</code>	No real root means a negative discriminant	quantified hypothesis	1.3 s	proved
<code>s_sqrt</code>	<code>sqrt(x)</code> becomes an unknown real	abstraction	2.6 s	proved
<code>s_fun</code>	An uninterpreted function, and a formula it cannot use	abstraction, skipping	1.8 s	proved
<code>s_alt_hyp</code>	Alternation inside a hypothesis	$\forall\exists$ hypothesis	1.5 s	proved
<code>s_mix</code>	Everything at once	abstraction, skipping	1.2 s	proved
<code>s_skip</code>	Formulas it has to skip	skipping	0.79 s	proved
<code>n_false</code>	A false sequent is left alone	FALSE	0.26 s	FALSE

(cad): one closed formula

Each lemma is proved by typing `(cad)` at the first prompt. The strategy prints one line, *Deciding 1 by cylindrical decision, witness first*, and PVS prints Q.E.D. when the proof is complete. The lines about length rewriting come from PVS's simplifier inside the strategy.

A circle meets a parabola

`d_meet`

$\exists\exists$, degree 2 · 3.8 s

The meeting points have $x^2 = (\sqrt{5} - 1)/2$, which is irrational, so there is no simple rational witness. The full cylindrical decision answers anyway: it computes with exact real algebraic numbers.

`d_meet: LEMMA EXISTS (x: real): EXISTS (y: real): x^2 + y^2 = 1 AND y = x^2`

```

d_meet :
  |-----
{1}  EXISTS (x: real): EXISTS (y: real): x ^ 2 + y ^ 2 = 1 AND y = x ^ 2

Rule? (cad)
length_singleton rewrites
  length[list[mpoly]]
    ((: (: mpol(: mconst(0), mconst(0), mconst(-1) :)), mconst(1) :) :))
  to 1
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time = 3.794 secs.
Real time = 3.797 secs.

```

The Motzkin polynomial is nonnegative

b_motzkin

$\forall\forall$, degree 6 · 1.1 s

The classic polynomial that is nonnegative everywhere but is *not* a sum of squares: a sum-of-squares certificate needs a multiplier ($x^2 + y^2 + 1$ times it is a sum of squares). A cylindrical decision needs no certificate of that kind.

b_motzkin: **LEMMA FORALL** (x: real): **FORALL** (y: real): $x^4*y^2 + x^2*y^4 - 3*x^2*y^2 + 1 \geq 0$

```

b_motzkin :
  |-----
{1}  FORALL (x: real):
      FORALL (y: real):
        x ^ 4 * y ^ 2 + x ^ 2 * y ^ 4 - 3 * x ^ 2 * y ^ 2 + 1 >= 0

Rule? (cad)
length_null rewrites length[list[mpoly]](: :)
  to 0
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time = 1.134 secs.
Real time = 1.135 secs.

```

The folium of Descartes reaches past (1,1)

b_folium

$\exists\exists$, degree 3 · 0.09 s

(cad) looks for a witness before deciding. A model search finds $x = y = 3/2$ and the proof is one witness point, instantiated in two steps, in 0.09 s. Section 5 shows the steps.

b_folium: **LEMMA EXISTS** (x: real): **EXISTS** (y: real): $x^3 + y^3 = 3*x*y$ **AND** $x > 1$ **AND** $y > 1$

```

b_folium :

|-----
{1}  EXISTS (x: real):
      EXISTS (y: real): x ^ 3 + y ^ 3 = 3 * x * y AND x > 1 AND y > 1

Rule? (cad)
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time  = 0.09 secs.
Real time = 0.09 secs.

```

The lemniscate fits in the disc of radius $\sqrt{2}$

b_lemniscate

$\forall\forall$, degree 4 · 1.1 s

An implication between two quartic conditions in two variables.

```

b_lemniscate: LEMMA FORALL (x: real): FORALL (y: real):
  (x^2 + y^2)^2 = 2*(x^2 - y^2) IMPLIES x^2 + y^2 <= 2

```

```

b_lemniscate :

|-----
{1}  FORALL (x: real):
      FORALL (y: real):
        (x ^ 2 + y ^ 2) ^ 2 = 2 * (x ^ 2 - y ^ 2) IMPLIES
        x ^ 2 + y ^ 2 <= 2

Rule? (cad)
length_singleton rewrites
length[list[mpoly]]
  ((: (: mpol(: mconst(-2), mconst(0), mconst(1) :)), mconst(0),
      mconst(1) :) :))
to 1
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time  = 1.10 secs.
Real time = 1.101 secs.

```

Every depressed cubic has a real root

c_cubic_root

$\forall\exists$, degree 3 · 0.76 s

Alternating quantifiers over a family of polynomials: for all coefficients p, q there is an x .

```

c_cubic_root: LEMMA FORALL (p: real): FORALL (q: real): EXISTS (x: real): x^3 + p*x + q = 0

```

```

c_cubic_root :

|-----

```

```
{1}  FORALL (p: real):
      FORALL (q: real): EXISTS (x: real): x ^ 3 + p * x + q = 0
```

Rule? (cad)

```
length_null rewrites length[list[mpoly]]((: :))
to 0
```

Deciding 1 by cylindrical decision, witness first,

Q.E.D.

Run time = 0.76 secs.

Real time = 0.76 secs.

The quadratic formula, as an existence theorem

d_quad_suf

$\forall\forall\exists$, degree 3 · 2.1 s

Four variables: for every $a \neq 0$, b, c with $b^2 \geq 4ac$ there is a root. No formula for the root is given; the decision establishes that one exists.

```
d_quad_suf: LEMMA FORALL (a: real): FORALL (b: real): FORALL (c: real): EXISTS (x: real):
  a /= 0 AND b^2 >= 4*a*c IMPLIES a*x^2 + b*x + c = 0
```

d_quad_suf :

```
|-----
{1}  FORALL (a: real):
      FORALL (b: real):
        FORALL (c: real):
          EXISTS (x: real):
            a /= 0 AND b ^ 2 >= 4 * a * c IMPLIES
              a * x ^ 2 + b * x + c = 0
```

Rule? (cad)

```
length_singleton rewrites
length[list[mpoly]]
  ((: (: mpol(: mconst(0), mconst(1) :)),
    mpol(: mpol(: mconst(0), mconst(1) :)) :)),
    mpol(: mpol(: mpol(: mconst(0), mconst(1) :)) :)) :)) :)) :))
to 1
```

Deciding 1 by cylindrical decision, witness first,

Q.E.D.

Run time = 2.108 secs.

Real time = 2.109 secs.

Wilkinson's polynomial of degree 20

h_wilk20

∀, degree 20 · 2.9 s

$(x-1)(x-2)\cdots(x-20)$ expanded, with coefficients up to 1.4×10^{19} . The arithmetic is exact rational arithmetic, so the famously ill-conditioned roots cause no trouble.

```
h_wilk20: LEMMA FORALL (x: real): x < 1 IMPLIES x^20 - 210*x^19 + 20615*x^18 - 1256850*x^17 +
  ↳ 53327946*x^16 - 1672280820*x^15 + 40171771630*x^14 - 756111184500*x^13 +
  ↳ 11310276995381*x^12 - 135585182899530*x^11 + 1307535010540395*x^10 -
  ↳ 10142299865511450*x^9 + 63030812099294896*x^8 - 311333643161390640*x^7 +
  ↳ 1206647803780373360*x^6 - 3599979517947607200*x^5 + 8037811822645051776*x^4 -
  ↳ 12870931245150988800*x^3 + 13803759753640704000*x^2 - 8752948036761600000*x +
  ↳ 2432902008176640000 > 0
```

h_wilk20 :

```
|-----
{1}  FORALL (x: real):
      x < 1 IMPLIES
      x ^ 20 - 210 * x ^ 19 + 20615 * x ^ 18 - 1256850 * x ^ 17 +
      53327946 * x ^ 16
      - 1672280820 * x ^ 15
      + 40171771630 * x ^ 14
      - 756111184500 * x ^ 13
      + 11310276995381 * x ^ 12
      - 135585182899530 * x ^ 11
      + 1307535010540395 * x ^ 10
      - 10142299865511450 * x ^ 9
      + 63030812099294896 * x ^ 8
      - 311333643161390640 * x ^ 7
      + 1206647803780373360 * x ^ 6
      - 3599979517947607200 * x ^ 5
      + 8037811822645051776 * x ^ 4
      - 12870931245150988800 * x ^ 3
      + 13803759753640704000 * x ^ 2
      - 8752948036761600000 * x
      + 2432902008176640000
      > 0
```

Rule? (cad)

length_singleton rewrites

length[list[mpoly]]

```
((: (: mconst(2432902008176640000), mconst(-8752948036761600000),
      mconst(13803759753640704000), mconst(-12870931245150988800),
      mconst(8037811822645051776), mconst(-3599979517947607200),
      mconst(1206647803780373360), mconst(-311333643161390640),
      mconst(63030812099294896), mconst(-10142299865511450),
      mconst(1307535010540395), mconst(-135585182899530),
      mconst(11310276995381), mconst(-756111184500),
      mconst(40171771630), mconst(-1672280820), mconst(53327946),
      mconst(-1256850), mconst(20615), mconst(-210), mconst(1) :) :))
```

to 1

Deciding 1 by cylindrical decision, witness first,**Q.E.D.**

Run time = 2.938 secs.

Real time = 2.94 secs.

Chebyshev T_{10} stays in $[-1, 1]$

h_cheb10

$\forall$, degree 10 · 2.0s

Two bounds joined by AND in the conclusion, on a polynomial that touches ± 1 eleven times in the interval.

h_cheb10: **LEMMA FORALL** (x: real): $-1 \leq x$ **AND** $x \leq 1$ **IMPLIES** $(512x^{10} - 1280x^8 + 1120x^6 - 400x^4 + 50x^2 - 1 \leq 1$ **AND** $512x^{10} - 1280x^8 + 1120x^6 - 400x^4 + 50x^2 - 1 \geq -1)$

```
h_cheb10 :
|-----
{1}  FORALL (x: real):
      -1 <= x AND x <= 1 IMPLIES
      (512 * x ^ 10 - 1280 * x ^ 8 + 1120 * x ^ 6 - 400 * x ^ 4 +
        50 * x ^ 2
        - 1
        <= 1
        AND
        512 * x ^ 10 - 1280 * x ^ 8 + 1120 * x ^ 6 - 400 * x ^ 4 +
        50 * x ^ 2
        - 1
        >= -1)

Rule? (cad)
length_singleton rewrites
length[list[mpoly]]
  ((: (: mconst(0), mconst(50), mconst(0), mconst(-400),
        mconst(0), mconst(1120), mconst(0), mconst(-1280), mconst(0),
        mconst(512) :) :))
to 1
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time = 2.023 secs.
Real time = 2.025 secs.
```

A root located to within 10^{-5}

g_mignotte1

$\exists$, degree 10 · 1.3s

$x^{10} = 2(100x - 1)^2$ has a root in $(0.00999, 0.01)$, a window 10^{-5} wide. Polynomials of this Mignotte type have roots packed very close together.

g_mignotte1: **LEMMA EXISTS** (x: real): $999/100000 < x$ **AND** $x < 1/100$ **AND** $x^{10} = 2*(100*x - 1)^2$

```
g_mignotte1 :
|-----
{1}  EXISTS (x: real):
```

```

999 / 100000 < x AND x < 1 / 100 AND x ^ 10 = 2 * (100 * x - 1) ^ 2

Rule? (cad)
length_singleton rewrites
  length[list[mpoly]]
    ((: (: mconst(-2), mconst(400), mconst(-20000), mconst(0), mconst(0),
          mconst(0), mconst(0), mconst(0), mconst(0), mconst(0),
          mconst(1) :) :))
  to 1
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time = 1.256 secs.
Real time = 1.256 secs.

```

The sign chart of $x^2 - 5x + 6$

m_quad_signs

$\forall$, degree 2 · 1.6 s

Three clauses joined by AND, with IMPLIES, OR and IFF inside: a whole sign table stated as one formula.

```

m_quad_signs: LEMMA FORALL (x: real): (x^2 - 5*x + 6 < 0 IMPLIES (2 < x AND x < 3)) AND (x^2 - 5*x
  ↪ + 6 > 0 IMPLIES (x < 2 OR x > 3)) AND (x^2 - 5*x + 6 = 0 IFF (x = 2 OR x = 3))

```

```

m_quad_signs :

|-----
{1}  FORALL (x: real):
      (x ^ 2 - 5 * x + 6 < 0 IMPLIES (2 < x AND x < 3)) AND
      (x ^ 2 - 5 * x + 6 > 0 IMPLIES (x < 2 OR x > 3)) AND
      (x ^ 2 - 5 * x + 6 = 0 IFF (x = 2 OR x = 3))

Rule? (cad)
length_singleton rewrites
  length[list[mpoly]]((: (: mconst(-2), mconst(1) :) :))
  to 1
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time = 1.561 secs.
Real time = 1.563 secs.

```

$x + y + z$ reaches 1.7 on the unit sphere

c_sphere_in

$\exists\exists\exists$, degree 2 · 11.6 s

The full decision is slow on this one, so (cad) fixes one coordinate to a small constant, decides the smaller sentence, and closes the goal by instantiation. The maximum is $\sqrt{3} \approx 1.732$.

```

c_sphere_in: LEMMA EXISTS (x: real): EXISTS (y: real): EXISTS (z: real):
  x^2 + y^2 + z^2 = 1 AND x + y + z = 17/10

```

```

c_sphere_in :
  |-----
{1}  EXISTS (x: real):
      EXISTS (y: real):
          EXISTS (z: real):
              x ^ 2 + y ^ 2 + z ^ 2 = 1 AND x + y + z = 17 / 10

Rule? (cad)
length_singleton rewrites
  length[list[mpoly]]
    ((: (: mpol(: mconst(-6/5), mconst(1) :)), mconst(1) :) :))
  to 1
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time = 11.577 secs.
Real time = 11.586 secs.

```

Three alternating quantifiers

t_forall_exists_forall

$\forall\exists\forall$, degree 4 · 22.9 s

For every x there is a y with $z^4 + yz^2 + y \geq x$ for every z . The slowest example here.

```

t_forall_exists_forall: LEMMA FORALL (x: real): EXISTS (y: real): FORALL (z: real): z^4 + y*z^2 +
  ↪ y >= x

```

```

t_forall_exists_forall :
  |-----
{1}  FORALL (x: real):
      EXISTS (y: real): FORALL (z: real): z ^ 4 + y * z ^ 2 + y >= x

Rule? (cad)
length_null rewrites length[list[mpoly]](: :))
  to 0
Deciding 1 by cylindrical decision, witness first,
Q.E.D.

Run time = 22.85 secs.
Real time = 22.867 secs.

```

Not every real has an inverse

d_no_inverse

$\forall\exists$ hypothesis · 0.54 s

PVS shows a negated goal as a hypothesis, so $\forall x \exists y : xy = 1$ is already formula -1 and (flatten) has nothing to do. (cad -1) decides that the hypothesis is FALSE (take $x = 0$), and a false hypothesis closes the goal.

```

d_no_inverse: LEMMA NOT (FORALL (x: real): EXISTS (y: real): x * y = 1)

```

```

d_no_inverse :

{-1} (FORALL (x: real): EXISTS (y: real): x * y = 1)
|-----

Rule? (flatten)
No change on: (FLATTEN)
d_no_inverse :

{-1} (FORALL (x: real): EXISTS (y: real): x * y = 1)
|-----

Rule? (cad -1)
length_null rewrites length[list[mpoly]]((: :))
to 0
Deciding -1 by cylindrical decision, witness first,
Q.E.D.

Run time = 0.54 secs.
Real time = 0.54 secs.

```

A false claim is reported, not proved

u_wilk8_val

$\exists$, degree 8 · 0.23 s

$(x-1)\cdots(x-8)$ never gets down to -1000 : its lowest values are about -641 , near $x \approx 1.31$ and 7.69 . (cad) says the sentence is FALSE and leaves it in place, labelled cad. The first comment on this lemma claimed TRUE; (cad) caught the mistake.

```

u_wilk8_val: LEMMA EXISTS (x: real):
  x^8 - 36*x^7 + 546*x^6 - 4536*x^5 + 22449*x^4 - 67284*x^3 + 118124*x^2 - 109584*x + 40320 = -1000

```

```

u_wilk8_val :

|-----
{1} EXISTS (x: real):
  x ^ 8 - 36 * x ^ 7 + 546 * x ^ 6 - 4536 * x ^ 5 + 22449 * x ^ 4 -
  67284 * x ^ 3
  + 118124 * x ^ 2
  - 109584 * x
  + 40320
  = -1000

Rule? (cad)
cad: the sentence is FALSE; it stays as it is, labelled cad
Deciding 1 by cylindrical decision, witness first,
this simplifies to:
u_wilk8_val : (CAD)

|-----
{1, cad}
EXISTS (x: real):
  x ^ 8 - 36 * x ^ 7 + 546 * x ^ 6 - 4536 * x ^ 5 + 22449 * x ^ 4 -
  67284 * x ^ 3
  + 118124 * x ^ 2

```

```

- 109584 * x
+ 40320
= -1000

```

Rule? (quit)

```

Run time  = 0.23 secs.
Real time = 0.23 secs.

```

(cad *): the whole sequent

Each lemma is first taken apart with (skeep) and (flatten), so that its facts sit in the sequent as formulas over skolem constants, next to formulas that are not about polynomials. (Here (skeep) already flattens, so (flatten) reports no change.) Then (cad *) says which formulas it used and skipped, which terms became unknown reals, and the sentence it decides.

No real root means a negative discriminant

s_disc

quantified hypothesis · 1.3s

The hypothesis is itself quantified. (cad *) keeps its quantifier and flips it: a $\forall$ in a hypothesis becomes an $\exists$ in the implication it decides.

s_disc: **LEMMA** **FORALL** (b, c: real): (**FORALL** (z: real): $z^2 + b * z + c > 0$) **IMPLIES** $b^2 < 4 * c$

s_disc :

```

|-----
{1}  FORALL (b, c: real):
      (FORALL (z: real):  $z^2 + b * z + c > 0$ ) IMPLIES  $b^2 < 4 * c$ 

```

Rule? (skeep)

Skolemizing and keeping names of the universal formula in (+ -),
this simplifies to:

s_disc : (SKEEP)

```

{-1}  FORALL (z: real):  $z^2 + b * z + c > 0$ 
|-----
{1}   $b^2 < 4 * c$ 

```

Rule? (flatten)

No change on: (FLATTEN)

s_disc : (SKEEP)

```

{-1}  FORALL (z: real):  $z^2 + b * z + c > 0$ 
|-----
{1}   $b^2 < 4 * c$ 

```

Rule? (cad *)

cad*: using (-1 1), skipping nothing; unknown reals: cadv_1 = b, cadv_2 = c

cad*: deciding **FORALL** (cadv_1: real): **FORALL** (cadv_2: real): **EXISTS** (cadv_b_1: real): ((((((cadv_b_1 ^ 2) +
→ (cadv_1 * cadv_b_1)) + cadv_2) > (0))) IMPLIES (((cadv_1 ^ 2) < ((4) * cadv_2))))

```

length_singleton rewrites
  length[list[mpoly]]
    ((: (: mpol(: mpol(: mconst(0), mconst(0), mconst(1) :)),
          mconst(-4) :)) :)) :))
  to 1
Deciding * by cylindrical decision, witness first,
Q.E.D.

Run time = 1.294 secs.
Real time = 1.295 secs.

```

sqrt(x) becomes an unknown real

s_sqrt

abstraction · 2.6s

$\sqrt{x}$ is not a polynomial, so (cad *) replaces it by a fresh real `cadv_2`. Because `sqrt` returns a nonnegative real, and `x` is declared nonnegative, it adds both facts as hypotheses of the sentence.

s_sqrt: **LEMMA FORALL** (x: nnreal, a: real): a = sqrt(x) **AND** a * a = x **AND** a > 1 **IMPLIES** x > 1

```

s_sqrt :

|-----
{1}  FORALL (x: nnreal, a: real):
      a = sqrt(x) AND a * a = x AND a > 1 IMPLIES x > 1

Rule? (skeep)
Skolemizing and keeping names of the universal formula in (+ -),
this simplifies to:
s_sqrt : (SKEEP)

{-1}  a = sqrt(x)
{-2}  a * a = x
{-3}  a > 1
|-----
{1}   x > 1

Rule? (flatten)
No change on: (FLATTEN)
s_sqrt : (SKEEP)

{-1}  a = sqrt(x)
{-2}  a * a = x
{-3}  a > 1
|-----
{1}   x > 1

Rule? (cad *)
cad*: using {-1 -2 -3 1}, skipping nothing; unknown reals: cadv_1 = a, cadv_2 = sqrt(x), cadv_3 = x
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): FORALL (cadv_3: real): (((cadv_2 >= 0) AND
    ↳ (cadv_3 >= 0)) IMPLIES (((cadv_1 = cadv_2) AND ((cadv_1 * cadv_1) = cadv_3) AND (cadv_1 > (1)))
    ↳ IMPLIES ((cadv_3 > (1)))))
length_singleton rewrites
  length[list[mpoly]]((: (: mconst(-1), mconst(1) :)) :))
  to 1
Deciding * by cylindrical decision, witness first,

```

Q.E.D.

Run time = 2.60 secs.
 Real time = 2.602 secs.

An uninterpreted function, and a formula it cannot use

s_fun

abstraction, skipping · 1.8 s

$f(x)$ and $\text{length}(l)$ become unknown reals. $\text{member}(x, l)$ is not about reals and is skipped.

```
s_fun: LEMMA FORALL (x: real, l: list[real]):
  f(x) > x^2 AND member(x, l) AND length(l) = 3 IMPLIES f(x) > -1
```

```
s_fun :

|-----
{1}  FORALL (x: real, l: list[real]):
      f(x) > x ^ 2 AND member(x, l) AND length(l) = 3 IMPLIES f(x) > -1

Rule? (skip)
Skolemizing and keeping names of the universal formula in (+ -),
this simplifies to:
s_fun : (SKEEP)

{-1}  f(x) > x ^ 2
{-2}  member(x, l)
{-3}  length(l) = 3
|-----
{1}  f(x) > -1

Rule? (flatten)
No change on: (FLATTEN)
s_fun : (SKEEP)

{-1}  f(x) > x ^ 2
{-2}  member(x, l)
{-3}  length(l) = 3
|-----
{1}  f(x) > -1

Rule? (cad *)
cad*: using (-1 -3 1), skipping (-2); unknown reals: cadv_1 = f(x), cadv_2 = x, cadv_3 = length(l)
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): FORALL (cadv_3: real): (((cadv_3 >= 0))
      ⇒ IMPLIES (((cadv_1 > (cadv_2 ^ 2)) AND (cadv_3 = (3))) IMPLIES ((cadv_1 > (-1)))))
length_singleton rewrites
  length[list[mpoly]]
    ((: (: mpol(: mpol(: mconst(1), mconst(1) :)) :)) :)) :))
  to 1
Deciding * by cylindrical decision, witness first,
Q.E.D.

Run time = 1.818 secs.
Real time = 1.819 secs.
```

Alternation inside a hypothesis

s_alt_hyp

$\forall \exists$ hypothesis · 1.5 s

The hypothesis $\forall z \exists w : w > z \wedge wc > 1$ becomes $\exists z \forall w$ in the implication. (cad *) builds that prenex form with renamed bound variables.

```
s_alt_hyp: LEMMA FORALL (c: real):
  (FORALL (z: real): EXISTS (w: real): w > z AND w * c > 1) IMPLIES c > 0
```

```
s_alt_hyp :

|-----
{1}  FORALL (c: real):
      (FORALL (z: real): EXISTS (w: real): w > z AND w * c > 1) IMPLIES
      c > 0

Rule? (skeep)
Skolemizing and keeping names of the universal formula in (+ -),
this simplifies to:
s_alt_hyp : (SKEEP)

{-1}  FORALL (z: real): EXISTS (w: real): w > z AND w * c > 1
|-----
{1}  c > 0

Rule? (flatten)
No change on: (FLATTEN)
s_alt_hyp : (SKEEP)

{-1}  FORALL (z: real): EXISTS (w: real): w > z AND w * c > 1
|-----
{1}  c > 0

Rule? (cad *)
cad*: using (-1 1), skipping nothing; unknown reals: cadv_1 = c
cad*: deciding FORALL (cadv_1: real): EXISTS (cadb_1: real): FORALL (cadb_2: real): (((cadb_2 > cadb_1)
    ↪ AND ((cadb_2 * cadv_1) > (1)))) IMPLIES ((cadv_1 > (0))))
length_singleton rewrites
  length[list[mpoly]]
    ((: (: mpol(: mpol(: mconst(0), mconst(1) :)) :)) :)) :))
  to 1
Deciding * by cylindrical decision, witness first,
Q.E.D.

Run time = 1.467 secs.
Real time = 1.468 secs.
```

Everything at once

s_mix

abstraction, skipping · 1.2 s

A predicate $P(x)$, a list fact, a quantified hypothesis about f , a polynomial fact and a disequality. Only the real-number content is used.

```
s_mix: LEMMA FORALL (x: real, l: list[real]): P(x) AND cons?(l) AND
  (FORALL (z: real): f(z) >= 0) AND f(x) > x^2 AND x /= 0 IMPLIES f(x) > 0
```

```
s_mix :
```

```
|-----
{1}  FORALL (x: real, l: list[real]):
      P(x) AND cons?(l) AND (FORALL (z: real): f(z) >= 0)
      AND (f(x) > x ^ 2) AND (x /= 0)
      IMPLIES f(x) > 0
```

Rule? (skeep)

Skolemizing and keeping names of the universal formula in (+ -),
this simplifies to:

```
s_mix : (SKEEP)
```

```
{-1}  P(x)
{-2}  cons?(l)
{-3}  (FORALL (z: real): f(z) >= 0)
{-4}  f(x) > x ^ 2
|-----
{1}   x = 0
{2}   f(x) > 0
```

Rule? (flatten)

No change on: (FLATTEN)

```
s_mix : (SKEEP)
```

```
{-1}  P(x)
{-2}  cons?(l)
{-3}  (FORALL (z: real): f(z) >= 0)
{-4}  f(x) > x ^ 2
|-----
{1}   x = 0
{2}   f(x) > 0
```

Rule? (cad *)

```
cad*: using (-4 1 2), skipping (-1 -2 -3); unknown reals: cadv_1 = f(x), cadv_2 = x
```

```
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): (((cadv_1 > (cadv_2 ^ 2))) IMPLIES ((cadv_2 =
  ↳ (0)) OR (cadv_1 > (0))))
```

length_singleton rewrites

```
length[list[mpoly]]((: (: mpol((: mconst(0), mconst(1) :)) :)) :))
to 1
```

Deciding * by cylindrical decision, witness first,

Q.E.D.

Run time = 1.207 secs.

Real time = 1.208 secs.

Formulas it has to skip

s_skip

skipping · 0.79 s

$\forall z: g(z) \leq 1$ has a bound variable under an unknown function, and the second hypothesis has a quantifier under IMPLIES. Neither is prenex polynomial, so both are skipped; $x > 2$ is enough.

```
s_skip: LEMMA FORALL (x: real): (FORALL (z: real): g(z) <= 1) AND
      (x > 0 IMPLIES EXISTS (z: real): z * z = x) AND x > 2 IMPLIES x > 1
```

```
s_skip :
```

```
  |-----
{1}  FORALL (x: real):
      (FORALL (z: real): g(z) <= 1) AND
      (x > 0 IMPLIES EXISTS (z: real): z * z = x) AND x > 2
      IMPLIES x > 1
```

Rule? (skeep)

Skolemizing and keeping names of the universal formula in (+ -),
this simplifies to:

```
s_skip : (SKEEP)
```

```
{-1}  (FORALL (z: real): g(z) <= 1)
{-2}  (x > 0 IMPLIES EXISTS (z: real): z * z = x)
{-3}  x > 2
      |-----
{1}  x > 1
```

Rule? (flatten)

No change on: (FLATTEN)

```
s_skip : (SKEEP)
```

```
{-1}  (FORALL (z: real): g(z) <= 1)
{-2}  (x > 0 IMPLIES EXISTS (z: real): z * z = x)
{-3}  x > 2
      |-----
{1}  x > 1
```

Rule? (cad *)

```
cad*: using (-3 1), skipping (-1 -2); unknown reals: cadv_1 = x
```

```
cad*: deciding FORALL (cadv_1: real): (((cadv_1 > (2))) IMPLIES ((cadv_1 > (1))))
```

length_singleton rewrites

```
  length[list[mpoly]]((: (: mconst(-1), mconst(1) :) :))
  to 1
```

Deciding * by cylindrical decision, witness first,

Q.E.D.

Run time = 0.79 secs.

Real time = 0.79 secs.

A false sequent is left alone

n_false

FALSE · 0.26 s

$x > 0 \vdash x > 1$ is false (take $x = 1/2$). (cad *) says so and the sequent is unchanged, because the whole attempt runs inside `finalize`, which backtracks on failure. (This lemma is in a scratch theory, since it is false.)

n_false: **LEMMA FORALL** (x: real): $x > 0$ **IMPLIES** $x > 1$

```
n_false :

  |-----
{1}  FORALL (x: real):  $x > 0$  IMPLIES  $x > 1$ 

Rule? (skeep)
Skolemizing and keeping names of the universal formula in (+ -),
this simplifies to:
n_false : (SKEEP)

{-1}   $x > 0$ 
  |-----
{1}   $x > 1$ 

Rule? (flatten)
No change on: (FLATTEN)
n_false : (SKEEP)

{-1}   $x > 0$ 
  |-----
{1}   $x > 1$ 

Rule? (cad *)
cad*: using (-1 1), skipping nothing; unknown reals: cadv_1 = x
cad*: deciding FORALL (cadv_1: real): (((cadv_1 > (0)))) IMPLIES ((cadv_1 > (1))))
cad: the sentence is FALSE; it stays as it is, labelled cad
cad*: not proved; the sequent is unchanged
No change on: (CAD *)
n_false : (SKEEP)

{-1}   $x > 0$ 
  |-----
{1}   $x > 1$ 

Rule? (quit)

Run time = 0.26 secs.
Real time = 0.26 secs.
```

Inside the strategy

Adding \$ to a strategy's name runs the same strategy but keeps each primitive step in the proof, so PVS prints the sequent after every step. This is what hides behind the one line of output above.

The witness route: (cad\$) on the folium

Before any decision, (cad) tries a model search for a point that settles the goal. For `b_folium` it finds $x = y = 3/2$: the goal is instantiated twice and evaluated.

```

b_folium :

|-----
{1}  EXISTS (x: real):
      EXISTS (y: real):  $x^3 + y^3 = 3 * x * y$  AND  $x > 1$  AND  $y > 1$ 

Rule? (cad$)
Labeling formula(s) (1) as (PAIRING !CWP_451),
this simplifies to:
b_folium : (RELABEL (:PAIRING !CWP_451) (1))

|-----
{1, !CWP_451}
      EXISTS (x: real):
        EXISTS (y: real):  $x^3 + y^3 = 3 * x * y$  AND  $x > 1$  AND  $y > 1$ 

No change on: (SKIP)
b_folium : (RELABEL (:PAIRING !CWP_451) (1))

|-----
{1, !CWP_451}
      EXISTS (x: real):
        EXISTS (y: real):  $x^3 + y^3 = 3 * x * y$  AND  $x > 1$  AND  $y > 1$ 

Instantiating the top quantifier in !CWP_451 with the terms:
  3/2,
this simplifies to:
b_folium : (INST !CWP_451 "3/2")

|-----
{1, !CWP_451}
      EXISTS (y: real):
         $(3 / 2)^3 + y^3 = 3 * (3 / 2) * y$  AND  $3 / 2 > 1$  AND  $y > 1$ 

Instantiating the top quantifier in !CWP_451 with the terms:
  3/2,
this simplifies to:
b_folium : (INST !CWP_451 "3/2")

|-----
{1, !CWP_451}
       $(3 / 2)^3 + (3 / 2)^3 = 3 * (3 / 2) * (3 / 2)$  AND
       $3 / 2 > 1$  AND  $3 / 2 > 1$ 

Evaluating formulas in *,
Q.E.D.

Run time  = 0.08 secs.
Real time = 0.09 secs.

```

The full decision: (cad-direct\$ 1) on $\exists x : x^2 = 2$

`t_sqrt2` has no rational witness, so the proof goes through the decision procedure. (cad-direct) is the decision half of (cad), without the witness search. Expanded, the proof has 1,023 lines of output; the excerpts below are consecutive runs of those lines, and every step of the proof falls in one of these stages. Recording and printing every step makes it slower: 3.4 s instead of 0.5 s.

1 The goal.

```
t_sqrt2 :
|-----
{1}   EXISTS (x: real): x ^ 2 = 2
```

2 Translate the sentence to data and run the decision. The quantifier prefix becomes `(: FALSE :)` (FALSE for $\exists$), the polynomial $x^2 - 2$ becomes its coefficient list `(: mconst(-2), mconst(0), mconst(1) :)`, and the matrix becomes `batom(0, 0)`: “polynomial number 0 has sign 0”. `eval-expr` runs `decide5` on this data in PVS’s ground evaluator, which is trusted (as are its other evaluations below, of normal forms), and adds the result as a hypothesis: the certificate passed (`ok`) and the sentence is true (`val`).

```
Labeling new formulas as !CDE_2,
this simplifies to:
t_sqrt2 : (DISCRIMINATE
  (EVAL-EXPR
    "decide5(reverse((: FALSE :)), (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))")
  !CDE_2)

{-1, !CDE_2}
  (decide5(reverse((: FALSE :)),
    (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0)))
  = (# ok := TRUE, val := TRUE #)
|-----
{1, !CDG_1}
  EXISTS (x: real): x ^ 2 = 2
```

3 Bring in the correctness theorem for exactly this data. `decide5_correct` says that whenever the certificate passes, `decide5` answers true exactly when the semantics `fsem` of the encoded sentence holds. It is instantiated with the three pieces of data.

```
Labeling new formulas as !CFS_6,
this simplifies to:
t_sqrt2 : (DISCRIMINATE (LEMMA "decide5_correct") !CFS_6)

{-1, !CFS_6}
  FORALL (F: list[list[mpoly]], os: list[bool], phi: BF):
    decide5(reverse(os), F, phi)`ok IMPLIES
      (decide5(reverse(os), F, phi)`val IFF fsem(os, F, phi, null))
{-2, !CDE_2}
  (decide5(reverse((: FALSE :)),
    (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0)))
  = (# ok := TRUE, val := TRUE #)
|-----
{1, !CDG_1}
  EXISTS (x: real): x ^ 2 = 2
```

```

Instantiating the top quantifier in !CFS_6 with the terms:
  (: (: mconst(-2), mconst(0), mconst(1) :) :) , (: FALSE :) , batom(0, 0) ,
this simplifies to:
t_sqrt2 : (INST !CFS_6 "(: (: mconst(-2), mconst(0), mconst(1) :) :) "
           "(: FALSE :) " "batom(0, 0)")

{-1, !CFS_6}
  decide5(reverse((: FALSE :)),
           (: (: mconst(-2), mconst(0), mconst(1) :) :) , batom(0, 0))`ok
  IMPLIES
  (decide5(reverse((: FALSE :)),
           (: (: mconst(-2), mconst(0), mconst(1) :) :) ,
           batom(0, 0))`val
  IFF
  fsem((: FALSE :) , (: (: mconst(-2), mconst(0), mconst(1) :) :) ,
        batom(0, 0), null))

[-2, !CDE_2]
  (decide5(reverse((: FALSE :)),
           (: (: mconst(-2), mconst(0), mconst(1) :) :) , batom(0, 0)))
  = (# ok := TRUE, val := TRUE #)
  |-----
[1, !CDG_1]
  EXISTS (x: real): x ^ 2 = 2

Case splitting on
  decide5(reverse((: FALSE :)), (: (: mconst(-2), mconst(0), mconst(1) :) :) ,

```

- 4 Split on ok and val**, drop the record, and simplify: the correctness theorem now says the encoded sentence holds. (The branches where ok or val is false contradict the evaluated record; see stage 9.)

```

Simplifying, rewriting, and recording with decision procedures,
this simplifies to:
t_sqrt2.1.1 : (ASSERT)

[-1] decide5(reverse((: FALSE :)),
           (: (: mconst(-2), mconst(0), mconst(1) :) :) , batom(0, 0))`val
[-2] decide5(reverse((: FALSE :)),
           (: (: mconst(-2), mconst(0), mconst(1) :) :) , batom(0, 0))`ok
{-3, !CFS_6}
  fsem((: FALSE :) , (: (: mconst(-2), mconst(0), mconst(1) :) :) ,
        batom(0, 0), null)
  |-----
[1, !CDG_1]
  EXISTS (x: real): x ^ 2 = 2

```

- 5 Unfold the semantics.** Each fsem step peels one quantifier off the data and puts a real quantifier in the formula; the last step turns the matrix into bfeval, the evaluation of the Boolean skeleton.

```

Expanding the definition of fsem,
this simplifies to:
t_sqrt2.1.1 : (EXPAND "fsem" !CFS_6)

[-1] decide5(reverse((: FALSE :)),
           (: (: mconst(-2), mconst(0), mconst(1) :) :) , batom(0, 0))`val
[-2] decide5(reverse((: FALSE :)),

```

```

      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-3, !CFS_6}
  EXISTS (x: real):
    fsem((: :), (: (: mconst(-2), mconst(0), mconst(1) :) :),
      batom(0, 0), cons(x, null))
  |-----
[1, !CDG_1]
  EXISTS (x: real): x ^ 2 = 2

```

Expanding the definition of fsem,
this simplifies to:

```

t_sqrt2.1.1 : (EXPAND "fsem" !CFS_6)

[-1] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-3, !CFS_6}
  EXISTS (x: real):
    bfeval(batom(0, 0), (: (: mconst(-2), mconst(0), mconst(1) :) :),
      cons(x, null))
  |-----
[1, !CDG_1]
  EXISTS (x: real): x ^ 2 = 2

```

6 Match the quantifiers. The existential hypothesis is skolemized with a constant x , and the goal's $\exists x$ is instantiated with the same x .

```

For the top quantifier in !CFS_6, we introduce Skolem constants: (x),
this simplifies to:
t_sqrt2.1.1 : (SKOLEM !CFS_6 ("x"))

[-1] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-3, !CFS_6}
  bfeval(batom(0, 0), (: (: mconst(-2), mconst(0), mconst(1) :) :),
    cons(x, null))
  |-----
[1, !CDG_1]
  EXISTS (x: real): x ^ 2 = 2

```

Instantiating the top quantifier in !CDG_1 with the terms:

```

x,
this simplifies to:
t_sqrt2.1.1 : (INST !CDG_1 "x")

[-1] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
[-3, !CFS_6]
  bfeval(batom(0, 0), (: (: mconst(-2), mconst(0), mconst(1) :) :),
    cons(x, null))
  |-----

```

```
{1, !CDG_1}
  x ^ 2 = 2
```

7 Unfold the skeleton down to one sign condition on the evaluated polynomial.

```
t_sqrt2.1.1 : (ASSERT)

[-1] decide5(reverse((: FALSE :)),
             (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-2] decide5(reverse((: FALSE :)),
             (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-3, !CFS_6}
  sign3(meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))
        (cons(x, null)))
    = 0
  |-----
[1, !CDG_1]
  x ^ 2 = 2
```

8 Reflection: the data is the polynomial. The strategy states that evaluating the coefficient list at x gives $x^2 - 2$, and proves it: the ground evaluator checks that normalizing the expression $x^2 - 2$ (pnorm) yields exactly this list, and the library lemma `peval_pnorm` says normalization preserves the value. After the replacement, the hypothesis is about $x^2 - 2$ itself.

```
Labeling new formulas as !CMV_8,
this yields 2 subgoals:
t_sqrt2.1.1.1 : (DISCRIMINATE
                (CASE
                  "meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) = ((x ^ 2)) - (2))"
                  !CMV_8)

{-1, !CMV_8}
  meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) =
    ((x ^ 2)) - (2)
{-2} decide5(reverse((: FALSE :)),
             (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
{-3} decide5(reverse((: FALSE :)),
             (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-4, !CFS_6}
  sign3(meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))
        (cons(x, null)))
    = 0
  |-----
[1, !CDG_1]
  x ^ 2 = 2
Replacing "meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :))" with "((x
                                                                    ^
                                                                    2))
                                                                    -
                                                                    (2)"

Replacing using formula !CMV_8,
this simplifies to:
t_sqrt2.1.1.1 : (REPLACE !CMV_8)

[-1, !CMV_8]
  meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) =
```

```

      ((x ^ 2)) - (2)
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :) , batom(0, 0))`val
[-3] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :) , batom(0, 0))`ok
{-4, !CFS_6}
      sign3(((x ^ 2)) - (2)) = 0
      |-----
[1, !CDG_1]
      x ^ 2 = 2

```

Labeling new formulas as !CMP_11,
this simplifies to:

t_sqrt2.1.1.2.1 : (DISCRIMINATE (LEMMA "peval_pnorm") !CMP_11)

```

{-1, !CMP_11}
  FORALL (e: PolyExpr, xs: list[real]):
    meval(pnorm(e))(xs) = peval(e)(xs)
{-2, !CMA_10}
  as_list(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2)))) =
    (: mconst(-2), mconst(0), mconst(1) :)
  |-----
[1, !CMV_8]
  meval(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2))))((: x :)) =
    ((x ^ 2)) - (2)

```

Instantiating the top quantifier in !CMP_11 with the terms:

pe_sub(pe_pow(pe_var(1), 2), pe_const(2)), (: x :),

this simplifies to:

t_sqrt2.1.1.2.1 : (INST !CMP_11 "pe_sub(pe_pow(pe_var(1), 2), pe_const(2))"
"(: x :)")

```

{-1, !CMP_11}
  meval(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2))))((: x :)) =
    peval(pe_sub(pe_pow(pe_var(1), 2), pe_const(2))))((: x :))
[-2, !CMA_10]
  as_list(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2)))) =
    (: mconst(-2), mconst(0), mconst(1) :)
  |-----
[1, !CMV_8]
  meval(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2))))((: x :)) =
    ((x ^ 2)) - (2)

```

Replacing "meval(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2))))((: x :))" with

```

  ↪ "peval(pe_sub(pe_pow(pe_var(1),
                                2),
                                pe_const(2))))
    ((: x :))"

```

Replacing using formula !CMP_11,

this simplifies to:

t_sqrt2.1.1.2.1 : (REPLACE !CMP_11)

```

[-1, !CMP_11]
  meval(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2))))((: x :)) =
    peval(pe_sub(pe_pow(pe_var(1), 2), pe_const(2))))((: x :))
[-2, !CMA_10]
  as_list(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2)))) =
    (: mconst(-2), mconst(0), mconst(1) :)

```

```

|-----
{1, !CMV_8}
  peval(pe_sub(pe_pow(pe_var(1), 2), pe_const(2)))((: x :)) =
    ((x ^ 2)) - (2)

,

This completes the proof of t_sqrt2.1.1.2.1.

t_sqrt2.1.1.2.2 : (DISCRIMINATE
  (CASE
    "as_list(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2)))) = (: mconst(-2),
      ↪ mconst(0), mconst(1) :)"
    !CMA_10)

|-----
{1, !CMA_10}
  as_list(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2)))) =
    (: mconst(-2), mconst(0), mconst(1) :)
{2, !CMV_8}
  meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) =
    ((x ^ 2)) - (2)

Keeping !CMA_10 and hiding *,
this simplifies to:
t_sqrt2.1.1.2.2 : (HIDE-ALL-BUT !CMA_10)

|-----
[1, !CMA_10]
  as_list(pnorm(pe_sub(pe_pow(pe_var(1), 2), pe_const(2)))) =
    (: mconst(-2), mconst(0), mconst(1) :)

Evaluating formula !CMA_10,

This completes the proof of t_sqrt2.1.1.2.2.

```

- 9 Finish.** `sign3` is unfolded (the zero case is already gone, because the goal $x^2 = 2$ counts as assumed false while it is being proved), the IF is lifted, and propositional reasoning closes the branch. The two side branches, where `ok` or `val` would be false, contradict the evaluated record and close by simplification.

```

Expanding the definition of sign3,
this simplifies to:
t_sqrt2.1.1.1 : (EXPAND "sign3" !CFS_6)

[-1] meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) =
      ((x ^ 2)) - (2)
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-3] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-4, !CFS_6}
  IF ((x ^ 2)) - (2) > 0 THEN 1 ELSE -1 ENDIF = 0
|-----
[1, !CDG_1]
  x ^ 2 = 2

Lifting IF-conditions to the top level,

```

this simplifies to:

`t_sqrt2.1.1.1 : (LIFT-IF !CFS_6)`

```
[-1] meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) =
      ((x ^ 2)) - (2)
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-3] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-4, !CFS_6}
  IF ((x ^ 2)) - (2) > 0 THEN 1 = 0 ELSE -1 = 0 ENDIF
  |-----
[1, !CDG_1]
  x ^ 2 = 2
```

No change on: (LIFT-IF !CFS_6)

`t_sqrt2.1.1.1 : (LIFT-IF !CFS_6)`

```
[-1] meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) =
      ((x ^ 2)) - (2)
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-3] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-4, !CFS_6}
  IF ((x ^ 2)) - (2) > 0 THEN 1 = 0 ELSE -1 = 0 ENDIF
  |-----
[1, !CDG_1]
  x ^ 2 = 2
```

No change on: (SKIP)

`t_sqrt2.1.1.1 : (LIFT-IF !CFS_6)`

```
[-1] meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) =
      ((x ^ 2)) - (2)
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-3] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-4, !CFS_6}
  IF ((x ^ 2)) - (2) > 0 THEN 1 = 0 ELSE -1 = 0 ENDIF
  |-----
[1, !CDG_1]
  x ^ 2 = 2
```

Postponing `t_sqrt2.1.1.1`.

`t_sqrt2.1.1.1 : (LIFT-IF !CFS_6)`

```
[-1] meval(mpol((: mconst(-2), mconst(0), mconst(1) :)))((: x :)) =
      ((x ^ 2)) - (2)
[-2] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[-3] decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
{-4, !CFS_6}
  IF ((x ^ 2)) - (2) > 0 THEN 1 = 0 ELSE -1 = 0 ENDIF
  |-----
```

```
[1, !CDG_1]
  x ^ 2 = 2
```

Applying propositional simplification and decision procedures,

This completes the proof of t_sqrt2.1.1.1.

This completes the proof of t_sqrt2.1.1.

```
t_sqrt2.1.2 : (CASE
  "decide5(reverse((: FALSE :)), (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0,
    ↪ 0))`val")

[-1] decide5(reverse((: FALSE :)),
  (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
[-2, !CFS_6]
  decide5(reverse((: FALSE :)),
    (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
  IMPLIES
  (decide5(reverse((: FALSE :)),
    (: (: mconst(-2), mconst(0), mconst(1) :) :),
      batom(0, 0))`val
    IFF
    fsem((: FALSE :), (: (: mconst(-2), mconst(0), mconst(1) :) :),
      batom(0, 0), null))
[-3, !CDE_2]
  (decide5(reverse((: FALSE :)),
    (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0)))
  = (# ok := TRUE, val := TRUE #)
  |-----
{1}  decide5(reverse((: FALSE :)),
  (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`val
[2, !CDG_1]
  EXISTS (x: real): x ^ 2 = 2
```

Simplifying, rewriting, and recording with decision procedures,

This completes the proof of t_sqrt2.1.2.

This completes the proof of t_sqrt2.1.

```
t_sqrt2.2 : (CASE
  "decide5(reverse((: FALSE :)), (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0,
    ↪ 0))`ok")

[-1, !CFS_6]
  decide5(reverse((: FALSE :)),
    (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
  IMPLIES
  (decide5(reverse((: FALSE :)),
    (: (: mconst(-2), mconst(0), mconst(1) :) :),
      batom(0, 0))`val
    IFF
    fsem((: FALSE :), (: (: mconst(-2), mconst(0), mconst(1) :) :),
      batom(0, 0), null))
[-2, !CDE_2]
  (decide5(reverse((: FALSE :)),
    (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0)))
```

```

      = (# ok := TRUE, val := TRUE #)
      |-----
{1}  decide5(reverse((: FALSE :)),
      (: (: mconst(-2), mconst(0), mconst(1) :) :), batom(0, 0))`ok
[2, !CDG_1]
      EXISTS (x: real): x ^ 2 = 2

```

Simplifying, rewriting, and recording with decision procedures,

This completes the proof of t_sqrt2.2.

Q.E.D.

Run time = 3.401 secs.

Real time = 3.404 secs.

The whole sequent: (cad\$ *) on the discriminant

The two usable formulas get labels, and the combined sentence G is introduced with case. On the first branch G is a hypothesis: it is instantiated with b and c , its $\exists$ is skolemized as cadk_1 , the sequent's own $\forall z$ is instantiated with cadk_1 , and propositional reasoning closes it. On the second branch G is the goal, and (cad) proves it. Both branches are checked by PVS, so nothing about the construction of G has to be trusted.

```

s_disc :

|-----
{1}  FORALL (b, c: real):
      (FORALL (z: real): z ^ 2 + b * z + c > 0) IMPLIES b ^ 2 < 4 * c

Rule? (skeep)
Skolemizing and keeping names of the universal formula in (+ -),
this simplifies to:
s_disc : (SKEEP)

{-1}  FORALL (z: real): z ^ 2 + b * z + c > 0
|-----
{1}  b ^ 2 < 4 * c

Rule? (flatten)
No change on: (FLATTEN)
s_disc : (SKEEP)

{-1}  FORALL (z: real): z ^ 2 + b * z + c > 0
|-----
{1}  b ^ 2 < 4 * c

Rule? (cad$ *)
cad*: using (-1 1), skipping nothing; unknown reals: cadv_1 = b, cadv_2 = c
cad*: deciding FORALL (cadv_1: real): FORALL (cadv_2: real): EXISTS (cadv_1: real): ((((((cadv_1 ^ 2) +
      ↪ (cadv_1 * cadv_1)) + cadv_2) > (0))) IMPLIES (((cadv_1 ^ 2) < ((4) * cadv_2))))
No change on: (SKIP)
s_disc :

[-1]  FORALL (z: real): z ^ 2 + b * z + c > 0
|-----

```

```

[1]   b ^ 2 < 4 * c

Using cadsf_465 to label formula(s) -1
this simplifies to:
s_disc : (LABEL |cadsf_465| -1)

{-1, cadsf_465}
  FORALL (z: real): z ^ 2 + b * z + c > 0
  |-----
[1]   b ^ 2 < 4 * c

Using cadsf_466 to label formula(s) 1
this simplifies to:
s_disc : (LABEL |cadsf_466| 1)

[-1, cadsf_465]
  FORALL (z: real): z ^ 2 + b * z + c > 0
  |-----
{1, cadsf_466}
  b ^ 2 < 4 * c

Labeling new formulas as cadsg_467,
this yields 2 subgoals:
s_disc.1 : (DISCRIMINATE
  (CASE
    "id(FORALL (cadv_1: real): FORALL (cadv_2: real): EXISTS (cadb_1: real): ((((((cadb_1 ^ 2) +
      ↪ (cadv_1 * cadb_1)) + cadv_2) > (0)))) IMPLIES (((cadv_1 ^ 2) < ((4) * cadv_2))))))"
    |cadsg_467|)

{-1, cadsg_467}
  id(FORALL (cadv_1: real):
    FORALL (cadv_2: real):
      EXISTS (cadb_1: real):
        ((((((cadb_1 ^ 2) + (cadv_1 * cadb_1)) + cadv_2) > (0))))
        IMPLIES (((cadv_1 ^ 2) < ((4) * cadv_2))))))

{-2, cadsf_465}
  FORALL (z: real): z ^ 2 + b * z + c > 0
  |-----
{1, cadsf_466}
  b ^ 2 < 4 * c

Expanding the definition of id,
this simplifies to:
s_disc.1 : (EXPAND "id" |cadsg_467| :ASSERT? NONE)

{-1, cadsg_467}
  FORALL (cadv_1: real):
    FORALL (cadv_2: real):
      EXISTS (cadb_1: real):
        ((((((cadb_1 ^ 2) + (cadv_1 * cadb_1)) + cadv_2) > (0))))
        IMPLIES (((cadv_1 ^ 2) < ((4) * cadv_2))))

[-2, cadsf_465]
  FORALL (z: real): z ^ 2 + b * z + c > 0
  |-----
[1, cadsf_466]
  b ^ 2 < 4 * c

Instantiating the top quantifier in cadsg_467 with the terms:

```

```

b,
this simplifies to:
s_disc.1 : (INST |cadsg_467| "b")

{-1, cadsg_467}
  FORALL (cadv_2: real):
    EXISTS (cadb_1: real):
      ((((((cadb_1 ^ 2) + (b * cadb_1)) + cadv_2) > (0))) IMPLIES
        (((b ^ 2) < ((4) * cadv_2))))
[-2, cadsf_465]
  FORALL (z: real): z ^ 2 + b * z + c > 0
  |-----
[1, cadsf_466]
  b ^ 2 < 4 * c

```

Instantiating the top quantifier in cadsg_467 with the terms:

```

c,
this simplifies to:
s_disc.1 : (INST |cadsg_467| "c")

{-1, cadsg_467}
  EXISTS (cadb_1: real):
    ((((((cadb_1 ^ 2) + (b * cadb_1)) + c) > (0))) IMPLIES
      (((b ^ 2) < ((4) * c))))
[-2, cadsf_465]
  FORALL (z: real): z ^ 2 + b * z + c > 0
  |-----
[1, cadsf_466]
  b ^ 2 < 4 * c

```

For the top quantifier in cadsg_467, we introduce Skolem constants: (cadk_1),
this simplifies to:

```

s_disc.1 : (SKOLEM |cadsg_467| ("cadk_1"))

{-1, cadsg_467}
  ((((((cadk_1 ^ 2) + (b * cadk_1)) + c) > (0))) IMPLIES
    (((b ^ 2) < ((4) * c))))
[-2, cadsf_465]
  FORALL (z: real): z ^ 2 + b * z + c > 0
  |-----
[1, cadsf_466]
  b ^ 2 < 4 * c

```

Instantiating the top quantifier in cadsf_465 with the terms:

```

cadk_1,
this simplifies to:
s_disc.1 : (INST |cadsf_465| "cadk_1")

[-1, cadsg_467]
  ((((((cadk_1 ^ 2) + (b * cadk_1)) + c) > (0))) IMPLIES
    (((b ^ 2) < ((4) * c))))
{-2, cadsf_465}
  cadk_1 ^ 2 + b * cadk_1 + c > 0
  |-----
[1, cadsf_466]
  b ^ 2 < 4 * c

```

Applying propositional simplification,

This completes the proof of s_disc.1.

```
s_disc.2 : (DISCRIMINATE
  (CASE
    "id(FORALL (cadv_1: real): FORALL (cadv_2: real): EXISTS (cadb_1: real): ((((((cadb_1 ^ 2) +
      ↪ (cadv_1 * cadb_1)) + cadv_2) > (0))) IMPLIES (((cadv_1 ^ 2) < ((4) * cadv_2))))))"
    |cadsg_467|)

{-1, cadsf_465}
  FORALL (z: real): z ^ 2 + b * z + c > 0
|-----
{1, cadsg_467}
  id(FORALL (cadv_1: real):
    FORALL (cadv_2: real):
      EXISTS (cadb_1: real):
        ((((((cadb_1 ^ 2) + (cadv_1 * cadb_1)) + cadv_2) > (0)))
          IMPLIES (((cadv_1 ^ 2) < ((4) * cadv_2))))))
{2, cadsf_466}
  b ^ 2 < 4 * c
```

Expanding the definition of id,
this simplifies to:

```
s_disc.2 : (EXPAND "id" |cadsg_467| :ASSERT? NONE)
```

```
[-1, cadsf_465]
  FORALL (z: real): z ^ 2 + b * z + c > 0
|-----
{1, cadsg_467}
  FORALL (cadv_1: real):
    FORALL (cadv_2: real):
      EXISTS (cadb_1: real):
        ((((((cadb_1 ^ 2) + (cadv_1 * cadb_1)) + cadv_2) > (0)))
          IMPLIES (((cadv_1 ^ 2) < ((4) * cadv_2))))))
[2, cadsf_466]
  b ^ 2 < 4 * c
```

```
length_singleton rewrites
length[list[mpoly]]
  ((: (: mpol(: mpol(: mconst(0), mconst(0), mconst(1) :)),
    mconst(-4) :)) :))
```

to 1

Deciding cadsg_467 by cylindrical decision, witness first,

This completes the proof of s_disc.2.

Q.E.D.

Run time = 1.283 secs.

Real time = 1.284 secs.

About this document

Source: the pvs_cad library, theories cad_demo, cad_limits, cad_limits2, cad_limits3 and cad_star_ex (each imports cad_decide5), plus a one-lemma scratch theory for the false (cad *) example. Traces captured on 28 September 2026 with PVS 8.1 and NASALib, from two scripted pvs -raw sessions. Cut from the transcripts: library loading, strategy-file

compilation and rewrite-rule installation messages, the Lisp REPL prompts, and the tag “Top: no parent rule” that PVS 8.1 appends to goal names in raw mode. Continuation lines of PVS’s “Replacing ... with ...” messages, indented by up to 100 columns, are indented by 64.